ПРАКТИЧЕСКАЯ РАБОТА №1

Тема: «Сравнение архитектуры операционных систем Windows NT, Linux и MINIX 3»

Дисциплина: …

Преподаватель: Семак Вячеслав Павлович

Выполнил: …

Группа: …

Дата выполнения: «__»_____ 20__ г.

2026

Оглавление

Введение.....3
Цель и задачи работы.....4
Средства выполнения работы.....6
Ход работы.....7
1. Выбор версий операционных систем.....7
2. Архитектурная организация исследуемых ОС.....8
3. Сравнение архитектур по единым критериям.....9
4. Роль размещения компонентов в обеспечении изоляции, устойчивости и безопасности.....11
5. Анализ ситуации компрометации драйвера.....12
Вывод.....13
Список использованных источников.....14

Введение

Любая операционная система (ОС) разделяет код на две части: ядро, работающее в привилегированном режиме процессора с полным доступом к оборудованию, и программы пространства пользователя, права которых ограничены. От того, какие компоненты включены в ядро, зависит тип архитектуры: монолитное ядро, микроядро или гибридное ядро.

Архитектурное решение определяет баланс между скоростью работы и надёжностью. Чем больше кода выполняется в ядре, тем выше производительность, но тем серьёзнее последствия ошибки или уязвимости в любом из компонентов. Для сравнения в работе выбраны три системы с разной архитектурой: Windows NT, Linux и MINIX 3.

Цель и задачи работы

Цель работы:

Получение практических и теоретических навыков в сравнительном анализе архитектуры операционных систем. В данной практической работе необходимо произвести сравнение архитектуры Windows NT, Linux и MINIX 3, определить расположение основных системных компонентов и проанализировать значение архитектурных решений для изоляции, устойчивости и информационной безопасности.

Задачи для выполнения работы:

1. Изучить техническую документацию по архитектуре Windows NT, Linux и MINIX 3, указав рассматриваемые версии систем и использованные источники.
2. Определить тип ядра каждой операционной системы и выделить особенности её архитектурной организации.
3. Установить назначение и расположение основных компонентов ОС: ядра, системных служб, драйверов, компонентов файловой системы, средств управления процессами и памятью.
4. Рассмотреть способы взаимодействия приложений, системных служб, драйверов и ядра, обозначив границы пространства пользователя и пространства ядра.
5. Составить упрощённые архитектурные схемы Windows NT, Linux и MINIX 3, отразив размещение компонентов и связи между ними.
6. Выполнить сравнительный анализ трёх ОС по единым критериям и представить результаты в сравнительной таблице.
7. Проанализировать влияние размещения и полномочий системных компонентов на их изоляцию, устойчивость ОС и информационную безопасность.

8. Рассмотреть ситуацию компрометации драйвера, определить возможные последствия для конфиденциальности, целостности и доступности и предложить меры снижения риска.

9. Сформулировать обоснованные выводы о сходствах, различиях и ограничениях исследованных архитектур.

10. Оформить отчёт, содержащий архитектурные схемы, сравнительную таблицу, разбор ситуации, выводы и список источников.

Средства выполнения работы

В процессе выполнения работы применялись:

- технические средства – компьютер с выходом в сеть Интернет;
- программные средства – веб-браузер, текстовый редактор и программа для просмотра PDF-документов;
- информационные средства – учебная литература [1], документация Microsoft [2–5], документация ядра Linux и дистрибутива Ubuntu [6–8], публикации и сайт проекта MINIX 3 [9–11].

Ход работы

1. Выбор версий операционных систем

Поскольку архитектура ОС меняется от версии к версии, для анализа были зафиксированы конкретные выпуски:

- Windows NT – Windows 11 версии 24H2, ядро NT 10.0, платформа x86-64 [2–5];
- Linux – дистрибутив Ubuntu 24.04 LTS, ядро версии 6.8, платформа x86-64 [6–8];
- MINIX 3 – выпуск 3.4.0, платформа x86 [9–11].

Сравнение систем выполнялось по одинаковым критериям, перечень которых приведён в разделе 3.

2. Архитектурная организация исследуемых ОС

Ядро Windows NT является гибридным. Формально система разделена на ядро, уровень аппаратных абстракций HAL и исполнительную систему ntoskrnl.exe, однако все они, а также драйверы и графическая подсистема Win32k.sys, работают в режиме ядра в одном адресном пространстве. В пространстве пользователя находятся приложения, системные службы и библиотека ntdll.dll, через которую выполняются системные вызовы [2–4].

Ядро Linux является монолитным с поддержкой модулей. Управление процессами и памятью, файловые системы, сетевой стек и драйверы устройств выполняются в общем адресном пространстве ядра, причём загружаемые модули (*.ko) после подключения получают те же права, что и само ядро. Приложения обращаются к ядру через системные вызовы, а также через /proc, /sys и ioctl [6].

MINIX 3 основана на микроядре, в котором оставлены только обработка прерываний, переключение процессов и обмен сообщениями. Драйверы устройств и системные серверы (PM, VM, VFS, RS и другие) работают как обычные процессы пространства пользователя и общаются между собой только сообщениями через ядро. Сервер реинкарнации RS следит за их работой и перезапускает компонент при отказе [9, 10].

3. Сравнение архитектур по единым критериям

Для сравнения систем использовались критерии, заданные в шаблоне работы. Результаты представлены в таблице (Таблица 1 – Сравнение архитектуры Windows NT, Linux и MINIX 3 по единым критериям).

Таблица 1 – Сравнение архитектуры Windows NT, Linux и MINIX 3 по единым критериям

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Рассматриваемая версия ОС	Windows 11, версия 24H2	Ubuntu 24.04 LTS (ядро 6.8)	MINIX 3.4.0
Название и тип ядра	NT (ntoskrnl.exe), гибридное ядро	Linux, монолитное ядро с модулями	MINIX 3, микроядро
Основные компоненты пространства ядра	Исполнительная система, ядро, HAL, Win32k.sys, драйверы устройств и ФС	Планировщик, подсистема памяти, VFS, файловые системы, сеть, драйверы	Только микроядро и задачи SYSTEM, CLOCK
Основные компоненты пространства пользователя	Приложения, службы, системные процессы (lsass, csrss), ntdll.dll	Приложения, библиотека glibc, systemd, системные службы	Приложения, системные серверы и драйверы устройств
Взаимодействие приложений, служб и ядра	Вызовы Win32 API через ntdll.dll и syscall; ALPC между процессами	Системные вызовы; /proc, /sys, ioctl	Передача сообщений через микроядро
Размещение драйверов устройств	В основном в ядре; часть – в UMDF	В ядре в виде модулей; отдельные – через FUSE, VFIO	В пространстве пользователя, по одному процессу на драйвер
Размещение файловых служб	Драйверы ФС (NTFS) в ядре	VFS и драйверы ФС (ext4) в ядре	Серверы VFS и MFS в пространстве пользователя
Управление процессами и памятью – основные особенности	Вытесняющее планирование потоков по приоритетам; виртуальная память со страничной организацией	Планировщик EEVDF, контрольные группы; виртуальная память, копирование при записи	Ядро выполняет переключение, политику задают серверы PM, SCHED и VM
Способы изоляции системных компонентов	Режимы процессора, отдельные адресные пространства процессов, VBS; компоненты ядра не изолированы	Режимы процессора, namespaces, seccomp, AppArmor; модули ядра не изолированы	Каждый компонент в своём адресном пространстве с ограниченными привилегиями

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Возможные последствия сбоя драйвера	Остановка системы («синий экран»)	Kernel panic или oops	Аварийное завершение одного процесса
Средства восстановления после отказа компонента	Перезагрузка, аварийный дамп, среда WinRE, перезапуск служб	Перезагрузка по panic=N, kdump, перезапуск служб systemd	Перезапуск отказавшего компонента сервером RS
Архитектурные особенности, значимые для безопасности	Подпись драйверов, PatchGuard, HVCI; большой объём привилегированного кода	Подпись модулей, режим lockdown, KASLR; большой объём привилегированного кода	Малый объём привилегированного кода; более низкая производительность

Конец Таблицы 1

Общим для всех систем является разделение режимов процессора и изоляция пользовательских процессов друг от друга. Основное различие заключается в том, где находятся драйверы, файловые системы и сетевые службы: в Windows NT и Linux они входят в ядро, а в MINIX 3 работают как отдельные процессы без привилегий ядра.

4. Роль размещения компонентов в обеспечении изоляции, устойчивости и безопасности

Код, выполняющийся в ядре, не ограничен никакими механизмами защиты ОС. Исходя из этого, влияние архитектуры можно описать по четырём направлениям:

- изоляция – в Windows NT и Linux защищена только граница между ядром и приложениями, тогда как в MINIX 3 изолирован каждый системный компонент;
- устойчивость – ошибка драйвера в Windows NT или Linux, как правило, останавливает всю систему, а в MINIX 3 ведёт лишь к перезапуску драйвера;
- безопасность – эксплуатация уязвимости в драйвере Windows или модуле Linux сразу даёт нарушителю полномочия ядра;
- производительность – отсутствие обмена сообщениями между компонентами ядра делает Windows NT и Linux быстрее MINIX 3.

Чтобы снизить риски, в Windows и Linux применяются проверка цифровой подписи драйверов и модулей, HVCI, режим lockdown и KASLR [5, 7], а часть драйверов переносится в пространство пользователя (UMDF, FUSE).

5. Анализ ситуации компрометации драйвера

В качестве ситуации рассмотрена эксплуатация уязвимости в драйвере сетевого адаптера: нарушитель отправляет на узел специально сформированные сетевые пакеты и добивается выполнения своего кода с полномочиями драйвера. Возможные последствия приведены в таблице (Таблица 2 – Последствия эксплуатации уязвимости сетевого драйвера для конфиденциальности, целостности и доступности).

Таблица 2 – Последствия эксплуатации уязвимости сетевого драйвера для конфиденциальности, целостности и доступности

Свойство информации	Windows NT	ОС на базе Linux	MINIX 3
Конфиденциальность	Доступ ко всей памяти, в том числе к учётным данным в процессе lsass.exe; перехват сетевого трафика	Доступ к памяти всех процессов и ключам шифрования; перехват сетевого трафика	Перехват только трафика, проходящего через сетевой драйвер
Целостность	Модификация ядра, отключение средств защиты, установка руткита	Модификация ядра, отключение AppArmor и аудита	Подмена сетевых пакетов; ядро и другие процессы недоступны
Доступность	Аварийная остановка системы	Kernel panic, остановка всех служб	Временная потеря сети до перезапуска драйвера

Таким образом, в Windows NT и Linux уязвимость одного драйвера приводит к полной компрометации узла, а в MINIX 3 ущерб ограничен функциями сетевого драйвера. Для снижения риска рекомендуются следующие меры:

1. регулярно обновлять ОС и сетевые драйверы;
2. использовать только подписанные драйверы и блокировать известные уязвимые [5, 7];
3. включить Secure Boot, а в Windows – VBS и HVCI;
4. отключить неиспользуемые устройства, включить IOMMU и фильтровать входящий трафик.

Вывод

В ходе выполнения практической работы были получены практические и теоретические навыки в сравнительном анализе архитектуры операционных систем Windows NT, Linux и MINIX 3.

Анализ показал, что Windows NT построена на гибридном ядре, Linux – на монолитном ядре с модулями, а MINIX 3 – на микроядре. Все три системы разделяют режимы процессора и изолируют пользовательские процессы, но различаются составом ядра. Компрометация драйвера в Windows NT и Linux затрагивает всю систему, тогда как в MINIX 3 её последствия ограничены одним компонентом. Монолитная и гибридная архитектуры обеспечивают более высокую производительность, но требуют дополнительных механизмов защиты, а микроядерная архитектура повышает изоляцию и устойчивость за счёт снижения скорости работы.

Список использованных источников

1. Таненбаум Э., Бос Х. Современные операционные системы. – 4-е изд. – СПб. : Питер, 2015. – 1120 с.
2. Yosifovich P., Ionescu A., Russinovich M. E., Solomon D. A. Windows Internals. Part 1. – 7th ed. – Redmond : Microsoft Press, 2017. – 800 p.
3. Overview of Windows Components [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/overview-of-windows-components (дата обращения: 09.11.2026).
4. User mode and kernel mode [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode (дата обращения: 09.11.2026).
5. Microsoft recommended driver block rules [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/microsoft-recommended-driver-block-rules (дата обращения: 09.11.2026).
6. The Linux Kernel documentation [Электронный ресурс]. – URL: https://docs.kernel.org/ (дата обращения: 09.11.2026).
7. Kernel module signing facility [Электронный ресурс] // The Linux Kernel documentation. – URL: https://docs.kernel.org/admin-guide/module-signing.html (дата обращения: 09.11.2026).
8. Ubuntu 24.04 LTS (Noble Numbat) release notes [Электронный ресурс] // Ubuntu Community Hub. – URL: https://discourse.ubuntu.com/t/ubuntu-24-04-lts-noble-numbat-release-notes/39890 (дата обращения: 09.11.2026).
9. Tanenbaum A. S., Woodhull A. S. Operating Systems: Design and Implementation. – 3rd ed. – Upper Saddle River : Prentice Hall, 2006. – 1054 p.

10. Herder J. N., Bos H., Gras B., Homburg P., Tanenbaum A. S. MINIX 3: A Highly Reliable, Self-Repairing Operating System // ACM SIGOPS Operating Systems Review. – 2006. – Vol. 40, No. 3. – P. 80–89.

11. MINIX 3 [Электронный ресурс] : официальный сайт проекта. – URL: https://www.minix3.org/ (дата обращения: 09.11.2026).